

Writing Drivers in RIO – An Overview

Introduction

With the Russound RIO protocol you can write a driver to control any RIO-enabled Russound product or even control an entire Russound audio system using your own control platform or user interfaces. By fully integrating a Russound audio system into a home control system, installation professionals are able to pair audio functions with other home automation features such as Lighting Scenes to create a rich user experience.

Here are some examples of what this robust protocol allows you to incorporate into a 3rd party system:

- Complete control of any zone, e.g. turn a zone on or off, adjust volume, adjust bass/treble, set loudness, link zones to a single source (party mode), etc.
- Select and control any source connected to the Russound system via IR or Russound's RNET serial connection – You don't have to know the IR commands or anything about RNET, the controller does all of this automatically through RIO.
- Access up to 32 global audio favorites per system (favorites are the easiest way to use streaming audio)
- Access up to 2 zone-specific audio favorites
- Support rich media content browsing and selection including full metadata and album art for Russound media sources that support them (streamers and tuners)

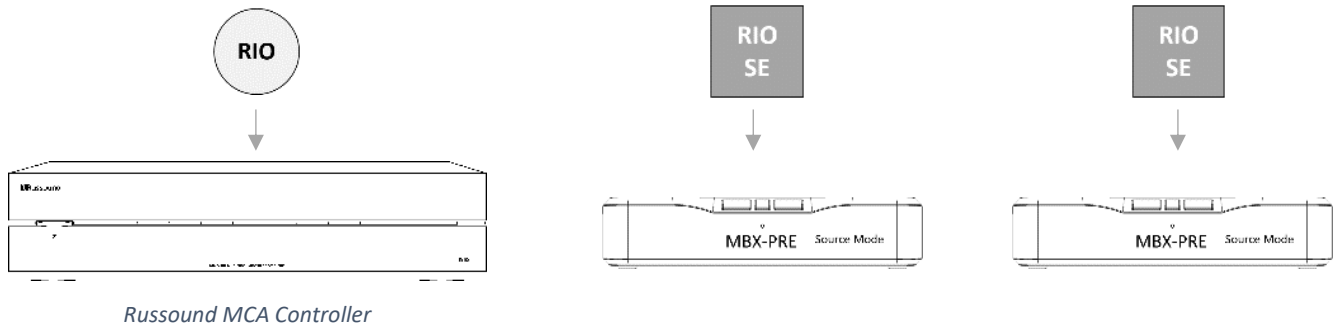
This document is designed to provide a brief overview to the RIO protocol and provides simple examples of some basic control functions and feedback messages used to offer insight into writing drivers to control a Russound device.

The document *RIO Protocol for 3rd Party Integrators* goes into detail of the structure of the full RIO protocol as well as provides details on menu navigation, Russound Streamer controls and many other features of RIO.

There is a companion document called *RIO Source Edition Protocol for 3rd Party Integrators* (RIO SE) which is a sister protocol to standard RIO that is used for certain Russound audio source devices as detailed later in this document. The most relevant of these products today is the MBX-PRE when used in Source Mode with a Russound controller. The typical way to control an MBX-PRE in Source Mode is through RIO SE. When using a Russound Controller with one or more MBX-PRE devices, the MBX-PRE devices must be in Source Mode for Russound User Interfaces to work properly (Keypads, Touchscreens, and the MyRussound app).

RIO and RIO Source Edition (RIO SE) Examples

Scenario #1: An MCA Controller System with MBX-PRE(s) as streaming sources



MCA Controller Functions	MBX-PRE Functions (Source Mode)
Volume Up/Down	Content Browsing and Selection
Volume Mute	
System Favorites	
Audio Settings (Bass, Treble, Loudness)	
Alarms, Sleep Timer and Schedules	

In this configuration, a driver written in RIO will be used for the Russound MCA Controller, and a driver written in RIO SE will be used for each MBX-PRE that is in Source Mode and used as an input to the MCA Controller. Each requires its own IP connection and sockets will be opened to each MBX device independently.

The MCA Controller RIO driver is recommended to have the following functions:

- Zone On
- Zone Off
- Zone Volume Up
- Zone Volume Down
- Zone Volume Mute Toggle
- Zone Volume Mute On
- Zone Volume Mute Off
- Set Zone Volume Level
- Get Zone Volume Level
- Select Source
- System Favorite Management - Restore (Play) a Favorite
 - Get Favorites

- Delete Favorites
- Save a Favorite
- Set/Get Bass, Treble, Loudness
- Party Mode On
- Party Mode Off
- All Zones Off
- Alarms/Schedules/Sleep Timer
 - Get Alarms/Schedules/Sleep Timer Status
 - Set Sleep Timer
 - Edit Sleep Timer
 - Edit Alarms/Schedules
 - Delete Alarms/Schedules

The MBX-PRE (Source Mode) RIO SE driver is recommended to have the following functions:

- Source Select (Selection of streaming sources and/or digital/analog inputs)
- Menu Navigation
- Metadata for all streaming sources
- Transport Controls for all streaming sources
- Streaming Service Indication, for audio that is cast to the MBX-PRE, ie TIDAL, Deezer
- Text Entry (for User Account info and for Searches)

Note: The above scenario assumes that the controller driver will have a limited subset of control options and that all source devices will be fully controlled independently and not through the controller. If instead you are using the Russound controller to fully control all connected source devices, then a much larger list of functions may be required. See the full RIO specification for KEY EVENTS, SETS, and ADJUSTS that may be needed.

Programming Note for RIO SE – The communication protocol for RIO SE requires knowing which Source Number has been configured for each connected Russound source. A driver parameter must allow for this source number to be configurable.

Scenario #2: An MBX System using only MBX-AMP or MBX-PRE (Zone Mode) devices



In this configuration, a driver written in RIO will be used for all of the Russound MBX devices. Each requires its own IP connection and sockets will be opened to each MBX device independently.

The RIO driver for this configuration is recommended to have the following functions:

- Volume Up
- Volume Down
- Volume Mute Toggle
- Volume Mute On
- Volume Mute Off
- Set Volume Level
- Get Volume Level
- System Favorite Management - Restore (Play) a Favorite
 - Get Favorites
 - Delete Favorites
 - Save a Favorite
- Set/Get Bass, Treble, Loudness
- All Zones Off
- Zone Favorite Management - Restore (Play) a Favorite
 - Get Favorites
 - Delete Favorites
 - Save a Favorite
- Source Select (Selection of streaming sources and/or digital/analog inputs)
- Menu Navigation
- Metadata for all streaming sources
- Transport Controls for all streaming sources
- Streaming Service Indication, for audio that is cast to the MBX-PRE, ie TIDAL, Deezer
- Text Entry (for User Account info and for Searches)
- Alarms, Sleep Timer, and Schedules
 - Get Alarms/Schedules/Sleep Timer Status
 - Set Sleep Timer
 - Edit Sleep Timer
 - Edit Alarms/Schedules
 - Delete Alarms/Schedules

Overview of Russound Products

Controllers

Russound uses the term “controller” to describe a multizone amplifier that can control connected source devices using infrared commands. At their core, controllers are advanced audio matrix devices that can route

the audio from any selected source to any connected zone in a system. Current Russound controller models are listed in the table below:

Model	Description	Number of sources [Max]	Number of zones [Max]	Control Interfaces
MCA-66	6-Source, 6-Zone Controller Amplifier	6 [6]	6 [36]	IP, RS-232
MCA-88	8-Source, 8-Zone Controller Amplifier	8 [8]	8 [48]	IP, RS-232

Streamers

Russound uses the term “streamer” to describe an audio source that provides content from the internet or local network that the device is connected to. Streamers can include built-in amplification and even a built-in loudspeaker, but can also be pure source devices. When used as a source device, they are designed primarily to be used as a dedicated source for a Russound controller, but they can also be used as the streaming audio source for any audio system, regardless of brand. Some Russound streamers can operate either way, with a simple configuration change option in their software. The MBX-PRE is an example of a Russound streamer with both a Source Mode (uses RIO SE and is paired with a Russound controller that uses RIO) and a Zone Mode (uses RIO and works with any brand’s audio system as a source device).

Model	Description	Number of streamers [Max]	Number of Amplified Zone Outputs	Control Interfaces
MBX-PRE	Wi-Fi Streaming Audio Player	1 [32]	0	IP
MBX-AMP	Wi-Fi Streaming Zone Amplifier	1 [32]	1	IP

Russound Driver Assortment

For a complete assortment of Russound drivers for all current Russound products, there may be up to 4 total drivers required. Two for the controller models, and two for the streamer models (MBX-PRE will need 2 drivers, one for each operating mode). Depending on how the driver is written, it is possible to write fewer drivers that have configurable options for each possible operating mode or system setup. The majority of the code written for each of the drivers will be identical across the models. While the current generation of Russound products are the models listed above, if the driver is written properly to ask for capabilities of each product it communicates with, it is easily possible to write a driver for current Russound devices which also works fine for any legacy MCA-C3, MCA-C5, or XStream device like the previous Russound streamer, the XSource.

Russound MCA-Series controller models all use the same protocol and command structure with the key difference being the number of zones per chassis and the total number of zones available. RIO commands are sent with a C[X].Z[Y] format where “X” is the controller number that is being addressed and “Y” is the zone

number of that particular controller. For example, if you are communicating with controller 1, zone 3, the format to address it would be “C[1].Z[3]”. Controller 2, zone 6 would be “C[2].Z[6]” as another example. It is therefore quite easy to write a single driver that is merely tweaked slightly so that it works with all Russound controller models by setting the maximum number of zones per controller to 6 for the MCA-66 and to 8 for the MCA-88.

The same driver for the controllers is also very similar to several of the streamer drivers. For example, all of the streamers except the MBX-PRE in Source Mode each use a default of C[1].Z[1] for all commands. Because of this design, it is very easy to write a single streamer driver that can easily be adapted to cover all of Russound’s streamer models with only some minor tweaking.

Full Control of a Russound system

If a system is being designed that allows the Russound controller to handle all of the source control of all connected devices, you can easily write a controller driver that will handle ALL sources, including streamers and keep the design future-proof for controlling any other devices through the Russound controller itself. This is because the Russound controller would be configured by a Russound installer with the appropriate IR control commands for the source and the Russound controller would send commands to the source as needed.

This involves a more complex driver because it would be required to monitor the Russound controller to identify the currently connected sources and operating state and then adapt the UI for the device type of each configured source. For example, if Source 3 on a controller is configured as a DVD player, it will require a different button assortment/UI than if the source is a cable box, or a radio tuner. There is a limited subset of possible source types that are configurable in a Russound system. If the driver being developed can adapt to source type notifications from the Russound controller and modify the UI accordingly, the entire audio system can be easily controlled and will adapt dynamically to any newly changed sources that an installer configures in the Russound devices.

Many control systems prefer to handle things in a different manner, i.e. with the control system controlling all connected sources directly. To explain the difference, let’s use an example of integrating a Russound controller into an imaginary system we’ll call ACME. How will the source gear (CD Player, Cable Box, Apple TV, etc.) be controlled?

Will the Russound controller be configured to control these sources, or will the ACME system be controlling these sources directly? That is the important difference. If the Russound system controls them, the driver must be able to handle all available Russound controller source types and adapt for them dynamically.

If the ACME system will be controlling the source devices, each source device can be configured in ACME and the only features that will likely be used in the Russound controller driver will be such things as:

Volume Up/Down/Mute	Audio Adjustments per zone (Balance/Treble/Loudness)	Party Mode (Linking Zones to the Same Source)	Zone On/Off
Source Switching	System On/Off	All Zones On/Off	

In this scenario, if the MCA Controller will ever be used with Russound streamers connected to it as sources, then additional support for recalling, storing, editing, and playing Russound system favorites should be implemented on the Russound Controller.

Key things to consider

- 1) If using a 3rd party control system to control all source devices, the 3rd party control system **MUST** be the only control system used. This is because Russound user interfaces would have buttons displayed that result in no action when pressed. Russound Displays and Apps would likely have source and zone information not matching the 3rd party system, etc. All of this would be detrimental to the user experience.
- 2) If using the Russound system to control all source devices with a driver that dynamically adjusts to the Russound system, a 3rd party control system can easily adapt to any changes and a user would be free to use either control platform (3rd party UI or Russound keypad/app/touchscreen) to control their system.
- 3) The Screen State of a user interface needs to be driven by RIO notifications. Some RIO commands may result in a change of the Screen State. The four screen states of RIO are identified by the acronym MINT: **M**enu, **I**nf, **N**ow playing, **T**ext Entry. Notification of the initial Screen State occurs after the first EVENT command is issued. Refer to the Media Management Session section of the RIO Protocol for more information. The RIO Protocol Specification gives design references for these screen types:

Screen State	Screen Notification (MMScreen)
Menu	SourceMenuScreen
Info	SourceInfoScreen
Now Playing	SourceNowPlayingScreen
Text Entry	SourceTextEntryScreen

An example of the **Media Management** notification is given on page 10 of this document.

First steps with a Russound Controller

The first step in a new integration project is to configure the Russound system that will be controlled by a 3rd party driver. If the Russound controller will be controlling the attached sources, all sources must be fully configured within the Russound system. If the 3rd party system will be controlling the sources, the installer will still need to configure the Russound controller with the source names (to make future system service easier when making connections or changes) and the audio connection type for each source (some connections can select between analog, digital coax, and digital optical).

Additionally, zone names should be set up in the Russound system even if they are only duplicates of the names used in the 3rd party system. This is done to make things easier for an installer so that on a service call, if the user reports the “Kitchen” has an issue, the installer will know that “Kitchen” is Zone 4 on the Russound Controller when troubleshooting the Russound device. Of course, all changes like this can be automatically synchronized to the Russound system using RIO if the driver is written properly to do so.

RIO GUIDELINES

- RIO commands are made up of ASCII characters except for the terminating characters
- All RIO commands must be terminated with a <CR> (0x0D hex)
- All RIO responses are terminated with a <CR><LF> (0x0D 0x0A hex)
- RIO command responses are preceded with a single letter indicating a successfully processed command (S), an error message (E) or notification response (N)
- In this document, all RIO commands will be colored **Green**. All RIO responses will be colored **Orange**.

PRIMARY RIO COMMANDS

VERSION

The **VERSION** command is used to request the version of the RIO protocol running on the RIO server device. This is an easy first command to verify the communications connection with your equipment.

To request the RIO protocol version:

VERSION Command:

```
VERSION
```

VERSION Response:

```
S VERSION="1.16.01"
```

Note: The VERSION command displays the RIO version, not the Russound device's firmware version.

WATCH

The **WATCH** command enables a device to register to receive asynchronous messages from a zone, source or system.

Enabling a WATCH on a zone is particularly powerful. In addition to receiving notifications on zone parameters (such as status, volume, etc.), the RIO client will also receive notifications for changes to the current source and its parameters (such as songName, shuffleMode, etc).

The System WATCH command [WATCH SYSTEM ON] is an excellent way for a UI device to remain aware of system status, allowing devices to display current information with minimal communication or overhead. Each notification message is uniquely identified with a key string. This allows the UI device a way to identify relevant data, while filtering unneeded data. These UI device decisions are sometimes made on a screen-by-screen basis, or a stateful manner.

When the command is issued with the parameter set to 'ON', it will provide a snapshot of the system parameters in the requested category. Subsequent changes will be sent to the requesting device as their values change. These asynchronous change notifications will continue until the WATCH command is turned 'OFF' by the user or when the WATCH command expires (when the EXPIRESIN option is specified).

To watch for asynchronous changes on controller 1, zone 6:

WATCH Command:

```
WATCH C[1].Z[6] ON
```

WATCH Response:


```
S
N C[1].Z[6].status="ON"
N C[1].Z[6].volume="20"
N C[1].Z[6].bass="10"
N C[1].Z[6].treble="10"
N C[1].Z[6].balance="10"
N C[1].Z[6].loudness="OFF"
N C[1].Z[6].currentSource="2"
N C[1].Z[6].name="Zone 6"
N S[2].artist="The Beatles"
N S[2].album="Abbey Road"
N S[2].song="Come Together"
```

EVENT

The **EVENT** command sends an event from a zone.

Here are some examples for controller 1, zone 4:

EVENT Command:

```
EVENT C[1].Z[4]!ZoneOn
EVENT C[1].Z[4]!ZoneOff
EVENT C[1].Z[4]!KeyRelease Power
EVENT C[1].Z[4]!AllOn
EVENT C[1].Z[4]!AllOff
EVENT C[1].Z[4]!KeyPress VolumeUp
EVENT C[1].Z[4]!KeyPress VolumeDown
EVENT C[1].Z[4]!KeyRelease Pause
EVENT C[2].Z[1]!KeyRelease Previous
EVENT C[2].Z[1]!KeyRelease Next
```

EVENT Response (Note that of the EVENT commands listed above, all would have an EVENT response):

```
S
```

GET

The **GET** command returns the value of one or more system parameters.

To get the value of the current source of controller 1, zone 4

GET Command:

```
GET C[1].Z[4].currentSource
```

GET Response:

```
S C[1].Z[4].currentSource="1"
```

SET

The **SET** command is used to modify the value of one or more system parameters.

To set the value of the turn on volume for controller 1, zone 5:

SET Command:

```
SET C[1].Z[5].turnOnVolume="25"
```

SET Response:

```
S C[1].Z[5].turnOnVolume="25"
```

ADJUST

The **ADJUST** command is used to modify the relative value of one or more system parameters.

To increase the value for the turn on volume of controller 1, zone 3 currently set to a value of 20:

ADJUST Command:

```
ADJUST C[1].Z[3].turnOnVolume="+1"
```

ADJUST Response:

```
S C[1].Z[3].turnOnVolume="21"
```

Media Management Session (MM) Example

The following commands are called once to configure the Media Management session. Refer to the RIO Protocol Specification for more detail.

Zone Mode (For Controller 1, Zone 1)

```
EVENT C[1].Z[1]!MMVerbosity 2
S
N S[1].MMScreen="SourceNowPlayingScreen"

EVENT C[1].Z[1]!MMIndex ABSOLUTE
S

EVENT C[1].Z[1]!MMMaxItems 30
S
```

Source Mode (For a Russound Media Streamer configured as Source 4)

```
EVENT S[4]!MMVerbosity 2
S
N S[4].MMScreen="SourceNowPlayingScreen"

EVENT S[4]!MMIndex ABSOLUTE
S
```

```
EVENT S[4]!MMMaxItems 30
S
```

The following commands are called to open and close menu navigation.

Zone Mode

```
EVENT C[1].Z[1]!MMInit
S
N S[1].MMScreen="SourceMenuScreen"
...
...
EVENT C[1].Z[1]!MMClose
S
N S[1].MMScreen="SourceNowPlayingScreen"
```

Source Mode

```
EVENT S[4]!MMInit
S
N S[4].MMScreen="SourceMenuScreen"
...
...
EVENT S[4]!MMClose
S
N S[4].MMScreen="SourceNowPlayingScreen"
```

Other Important Media Management Commands

MMSelectOption

```
EVENT C[c].Z[z]!MMSelectOption default
```

This command will play last played selection when this service is selected from the top menu. It is only available when the top level menu service has a previously played selection available.

```
EVENT C[c].Z[z]!MMSelectOption norestore
```

This command will navigate into the service menu of the selected source.

MMSelectItem

Use MMSelectItem to select an item from the menu

```
EVENT C[c].Z[z]!MMSelectItem [1 to max items]
```

MMSelectItem can also be used to set an optional value when selected.

```
EVENT C[c].Z[z]!MMSelectItem [1 to max items] [<value>]
```

This is valid when used with menu items that have the following attributes (see RIO specification for attributes):

- Editable
- EditTypeString
- EditTypeIPAddress
- EditTypeNumber
- EditTypeSlider
- EditTypeRadioBox
- EditTypeBool

Example:

Here is an example of setting the text in the TuneIn Search field. The TuneIn Search menu includes the following attributes:

```
N S[1].MMMenuItem[1].attr="BELMFJes"
```

Attribute "e" indicates Editable

Attribute "s" indicates EditTypeString

```
EVENT C[1].Z[1]!MMSelectItem 1 "WROR"
```

RIO Protocol Examples

Basic integration

Our first driver example will be designing a simple IP control 7-button keypad for the kitchen that will provide zone volume up and down, selection of two zone specific favorites, the ability to go to the next available source, pause/mute of zone source audio and powering off the zone.

In our configuration, the kitchen is zone 3 on controller 1.

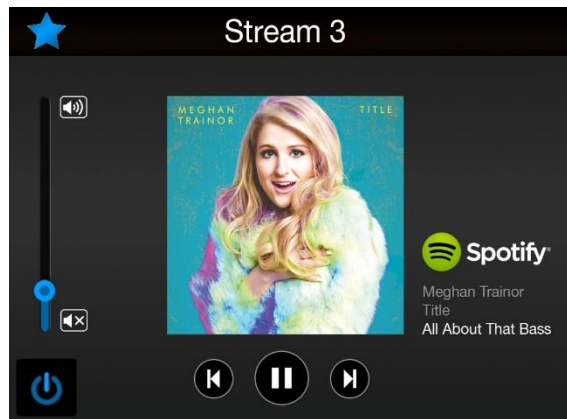


Sample Keypad Button	RIO Command
Zone Favorite 1	EVENT C[1].Z[3]!RestoreZoneFavorite 1
Zone Favorite 2	EVENT C[1].Z[3]!RestoreZoneFavorite 2
Source	EVENT C[1].Z[3]!KeyRelease NextSource
Pause/Mute	EVENT C[1].Z[3]!KeyRelease Pause
Volume Up	EVENT C[1].Z[3]!KeyPress VolumeUp
Volume Down	EVENT C[1].Z[3]!KeyPress VolumeDown
Power	EVENT C[1].Z[3]!ZoneOff

Intermediate Integration – Control, Feedback, and Metadata

In our previous example, most of our functionality was zone specific. In this example we will add in source functionality. We'll be using a Russound MBX-PRE as the music source. The MBX-PRE is connected directly to the Russound controller as source 3 and configured for "Source Mode".

In this driver example we are designing a touchscreen for the den that will provide zone power, volume up and down with a visual indicator, access to 32 global favorites, source name, metadata feedback, album artwork and source control for the Russound MBX-PRE. In our configuration, the den is zone 1 on controller 2 and the MBX-PRE (Source Mode) is assigned as source 3 on the controller.



Watch Command and Results	RIO Examples (Responses can be used for generating a UI)
Watch	<code>WATCH C[2].Z[1] ON</code>
Volume Indicator	<code>N C[2].Z[1].volume="volume level"</code>
Source Name	<code>N S[3].Name="Stream3"</code>
Mode	<code>N S[3].mode="Spotify"</code>
Metadata Artist	<code>N S[3].artistName="Meghan Trainor"</code>
Metadata Album	<code>N S[3].albumName="Title"</code>
Metadata Song	<code>N S[3].songName="All About That Bass"</code>
Album artwork	<code>N S[3].coverArtURL="http://o.scdn.co/640/9e9872320e69846dcefd936efd704e5e8f5a0bc7"</code>

UI Buttons and Controls	Associated RIO Command
Power	<code>EVENT C[2].Z[1]!KeyRelease Power</code>
Volume Up	<code>EVENT C[2].Z[1]!KeyPress VolumeUp</code>
Volume Down	<code>EVENT C[2].Z[1]!KeyPress VolumeDown</code>
Favorites Menu	<i>Suggestion: A press and release of the favorite icon "Star" can bring up a list of the 32 global favorites - See below for sample code regarding Favorites.</i>
Previous Track	<code>EVENT C[2].Z[1]!KeyRelease Previous</code>
Pause	<code>EVENT C[2].Z[1]!KeyRelease Pause</code>
Next Track	<code>EVENT C[2].Z[1]!KeyRelease Next</code>

Favorites Sample Code

A custom screen to display and create favorites can be generated using the following RIO commands:

Display favorites

For Favorites, one solution is to iterate through the list of 32 system favorites, then display the favorite if the entry is valid.

```
GET system.favorite[7].valid
S System.favorite[7].valid="TRUE"

GET system.favorite[7].name
S System.favorite[7].name="My Favorite #7"
```

Another solution is to use the WATCH SYSTEM function which will give notification of all system favorites and their status, preventing the need to iterate.

```
WATCH SYSTEM ON
```

Save favorite

```
EVENT C[1].Z[1]!saveSystemFavorite "SysFav#8" 8
S
```

Change the name of a favorite

```
SET system.favorite[8].name="My Favorite #8"
S System.favorite[8].name=" My Favorite #8"
```

Restore favorite

```
EVENT C[1].Z[1]!restoreSystemFavorite 8
S
```

Delete favorite

```
EVENT C[1].Z[1]!deleteSystemFavorite 8
S
```

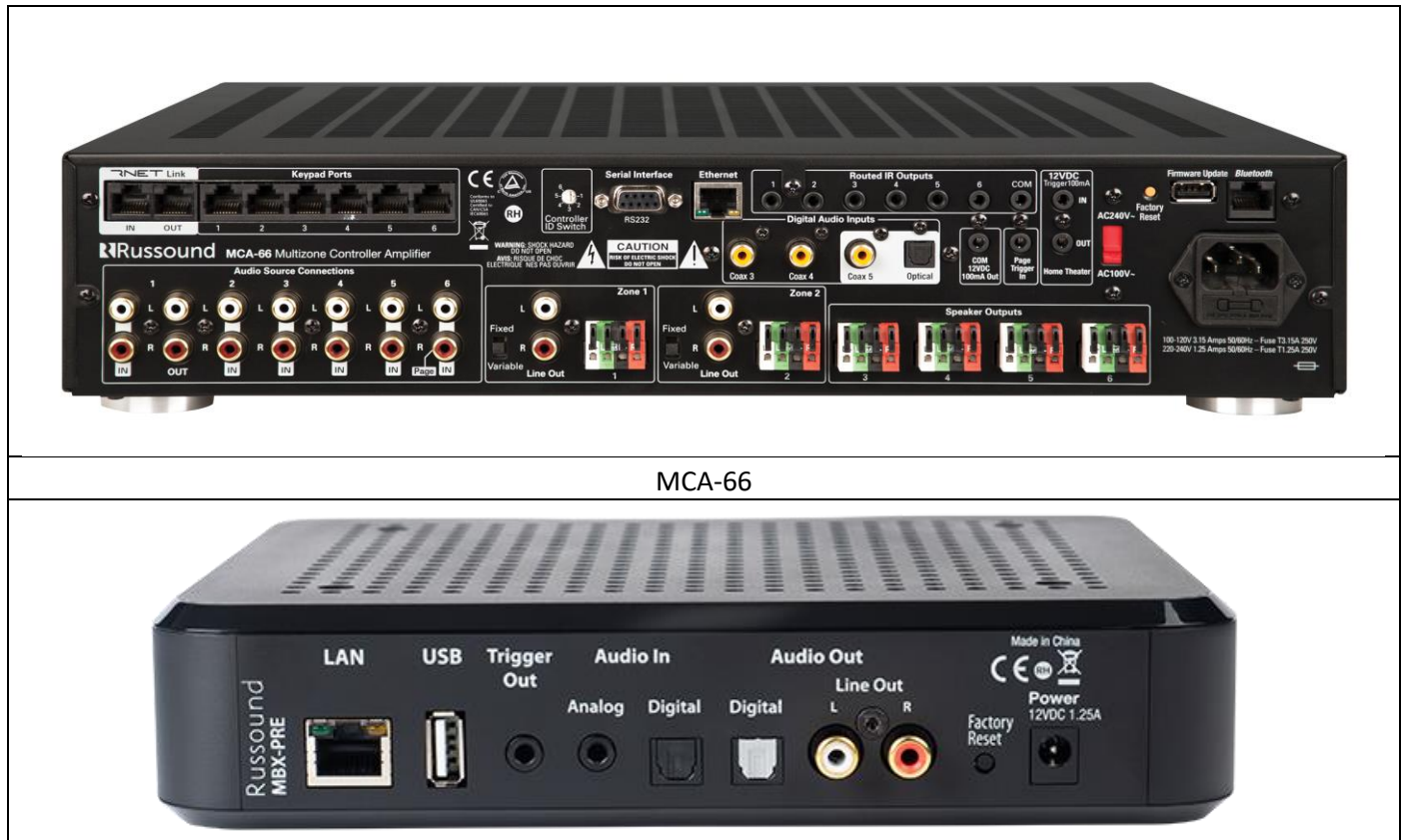
Note that the global “system” favorites reside on the Russound zone controller and not on any source devices such as an MBX-PRE in Source Mode. When using an MBX-PRE in Source Mode with a Russound controller, RIO SE must be used for the MBX-PRE while RIO must be used with the controller in order to access system favorites.

Appendix 1: Specific protocol and IP port connections

PRODUCT	PRODUCT TYPE	INTEGRATION DRIVER PROTOCOL	TCP PORT	Comments	MINIMUM FIRMWARE VERSION	Recommended Driver Name
MCA-66	MATRIX AUDIO SWITCHER WITH INFRARED AND RNET CONTROL OF CONNECTED AUDIO SOURCES (RUSSOUND CALLS THIS A "CONTROLLER")- 6 SOURCES WITH 6 ZONES, UP TO 6 OF THESE MAY BE LINKED IN A SINGLE SYSTEM FOR UP TO A 6 SOURCE/36 ZONE INSTALLATION.	RIO	9621		3.00.04	MCA66?.???
MCA-88	MATRIX AUDIO SWITCHER WITH INFRARED AND RNET CONTROL OF CONNECTED AUDIO SOURCES (RUSSOUND CALLS THIS A CONTROLLER) - 8 SOURCES WITH 8 ZONES, UP TO 6 OF THESE MAY BE LINKED IN A SINGLE SYSTEM FOR UP TO AN 8 SOURCE/48 ZONE INSTALLATION.	RIO	9621		3.00.04	MCA88?.???
MBX-PRE-Source Mode	AN MBX-PRE IS A PREAMPLIFIER AUDIO STREAMER. WHEN IN "SOURCE" MODE, IT IS DESIGNED TO BE USED AS A SOURCE WITH A RUSSOUND MCA-SERIES CONTROLLER AND CAN BE CONTROLLED EASILY BY A FULL CONTROLLER DRIVER. WHENEVER IT IS NECESSARY TO HAVE INDEPENDENT CONTROL OR A SEPARATE IP ADDRESS, USE THE RIO-SE PROTOCOL. IF NOT USING THE MBX-PRE WITH A CONTROLLER, I.E. AS A STAND-ALONE AUDIO STREAMER FOR USE WITH ANY AMPLIFIER/RECEIVER OF ANY BRAND, USE THE MBX-PRE in ZONE MODE (Uses the RIO Protocol)	RIO SE	9621	The MBX-PRE replaces the XSource	1.00.26	MBX-PRE SRC?.???

PRODUCT	PRODUCT TYPE	INTEGRATION DRIVER PROTOCOL	TCP PORT	Comments	MINIMUM FIRMWARE VERSION	Recommended Driver Name
MBX-PRE-Zone Mode	AN MBX-PRE IS A PREAMPLIFIER AUDIO STREAMER. WHEN IN "ZONE" MODE, IT IS DESIGNED TO BE USED AS A STAND-ALONE AUDIO STREAMER THAT CAN BE USED WITH ANY BRAND AMPLIFIER/RECEIVER. THE PROTOCOL IS DIFFERENT FOR ZONE MODE AND SOURCE MODE AS NOTED HERE	RIO	9621	The MBX-PRE replaces the XSource	1.00.26	MBX-PRE_ZONE.???
MBX-AMP	AN MBX-AMP IS AN AMPLIFIED AUDIO STREAMER. IT IS DESIGNED TO BE USED AS A STAND-ALONE AUDIO STREAMER. UP TO 32 OF THEM MAY BE USED IN A SINGLE INSTALLATION.	RIO	9621		1.00.26	MBX-AMP.???

Appendix 2: Rear Panel Images of some of our products to illustrate the connection possibilities



MBX-PRE



MBX-AMP